Page No. | Date

ex: subtract 0111000_2 (56_{10}) from 1011100_2 (92_{10}) using complimentary method.

complement of 0111000_2 = 1000111_2

```
  (1)(1)(1)
   1011100
 + 1000111
 ---------
(1)0100011
  |     + 1
  ↳
 ---------
  0100100
```

Result = $0100100_2 = 36_{10}$

ex: Subtract 100011_2 (35_{10}) from 010010_2 (18_{10})

```
  010010
+ 011100   (complement of 100011)
 -------
  101110
```

no carry, so recomplement and attach negative sign

$= -010001_2 = -17_{10}$

ex: Subtract 01110_2 from 10101_2 using complementary method.

```
  10101
+ 10001   (complement of 01110₂)
 -------
(1)00110
  |   +1
  ↳
 -------
```

result : 00111

Page No.
Date

multiplication

0 * 0 = 0
0 * 1 = 0
1 * 0 = 0
1 * 1 = 1

ex: multiply binary number 1010 and 1001

Sol:

```
    1 0 1 0
  * 1 0 0 1
  ---------
    1 0 1 0
  0 0 0 0 *
0 0 0 0 * *
1 0 1 0 * * *
-------------
1 0 1  1 0 1 0   : result
```

Shortcut:

```
    1 0 1 0
  * 1 0 0 1
  ---------
    1 0 1 0
1 0 1 0 S S   → because of two zero
-------------
1 0 1 1 0 1 0
```

ex: multiply binary number 1111 and 111.

```
      1 1 1 1
    (1)* 1 1 1
  ---------------
  (1)(1) 1 1 1 1
(1) 1 1 1 1 *
1 1 1 1 * *
-----------------
1 1 0 1 0 0 1
```

1+1 = 10
1+1+1 = 11
1+1+1+1 = 0 carry two.

Page No.
Date

additive method of multiplication

4 * 8 will be calculated as 8+8+8+8.

(repeated complementary addition)

Division.

0 ÷ 0 = divide by zero error

0 ÷ 1 = 0

1 ÷ 0 = divide by zero error.

1 ÷ 1 = 1

ex: divide 100001_2 by 110_2

```
        0101
   110 ) 100001
       - 110        → divisor greater than 100, put 0 in quotient
         1000       → add digit from dividend
       - 110        → subtraction possible, here put 1 in quotient
          100       → remainder from subtraction + digit dividend
        - 110       → divisor greater, here put 0 in quotient
          1001      → add digit from dividend
        - 110       → subtraction possible, put 1 in quotient
           11       remainder
```

quotient = 5_{10} , remainder = 3_{10}

additive method of division. (repeated complementary subtraction)

35 ÷ 5 will be calculated as

35 - 5 = 30
30 - 5 = 25
25 - 5 = 20
20 - 5 = 15
15 - 5 = 10
10 - 5 = 5
5 - 5 = 0

} total steps = 7
hence result = 7.

Page No.
Date

ex: divide 33_{10} by 6_{10} by method of addition.

Sol:

$33 - 6 = 27$
$27 - 6 = 21$
$21 - 6 = 15$
$15 - 6 = 9$
$9 - 6 = 3$
$3 - 6 = -3$

} no of steps = 5, quotient = 5.
remainder = 3

Most computer use additive approach for performing subtraction, multiplication and division operations.

Page No.
Date

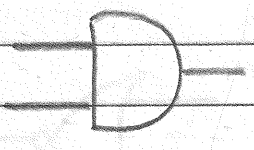
Logic gates

All operations within a computer are carried out by means of combinations of signals passing through standard blocks of built-in circuit known as logic gates.

A logic gate is an electronic circuit that operates on one or more input signals to produce standard output signals.

These logic gates are building blocks of all electronic circuits in a computer.

AND Gate (·)

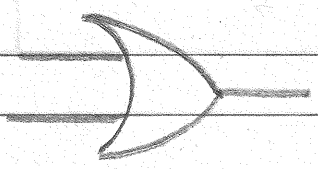


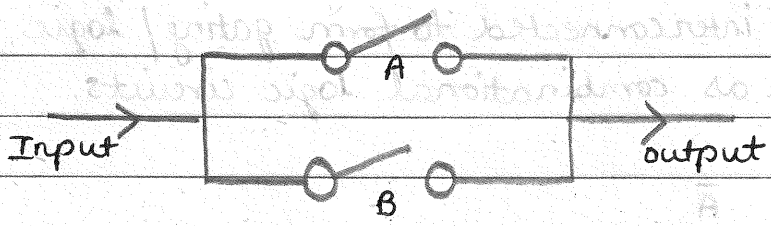


Two or more switches connected in series behave as an AND gate.

A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

OR Gate (+)



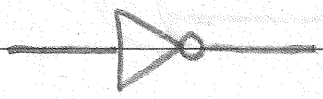


Two or more switches connected in parallel behave as an OR gate.

A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

NOT Gate (inverter) (¯)

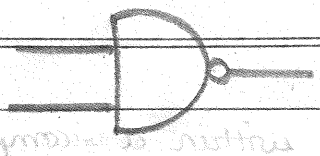
↳ because inverts input.



A	$\bar{A}$
0	1
1	0

Page No.
Date

NAND Gate. (↑)

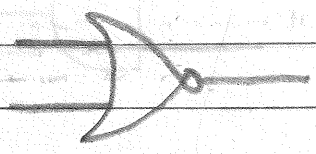


↳ complemented AND gate. (AND + NOT)

$A \uparrow B = \overline{A \cdot B} = \overline{A} + \overline{B}$

A	B	$\overline{A}+\overline{B}$
0	0	1
0	1	1
1	0	1
1	1	0

NOR Gate (↓)



↳ complemented OR gate (OR + NOT)

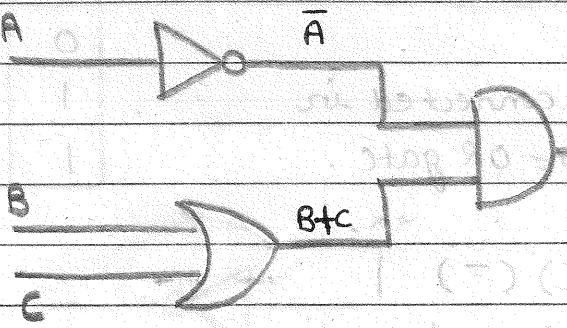
$A \downarrow B = \overline{A+B} = \overline{A} \cdot \overline{B}$

A	B	$\overline{A}\cdot\overline{B}$
0	0	1
0	1	0
1	0	0
1	1	0

→ Logic circuits

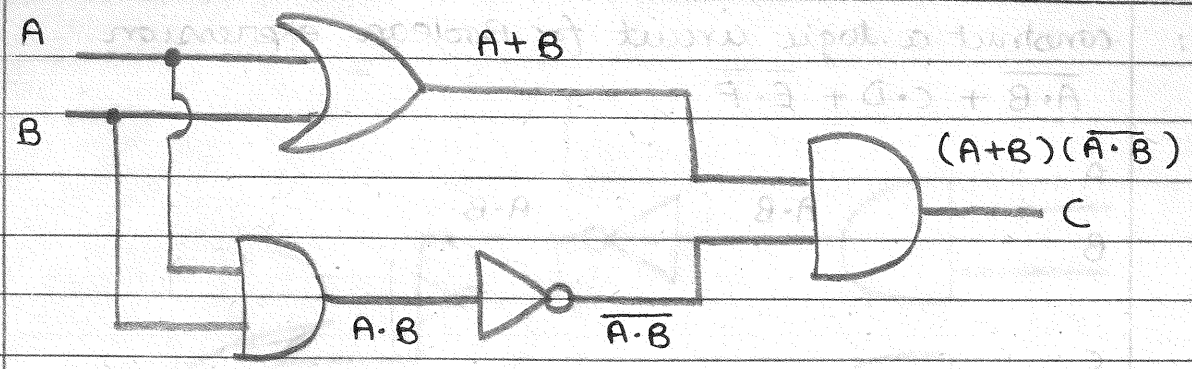
Logic gates are interconnected to form gating / logic network known as combinational logic circuits.

ex:

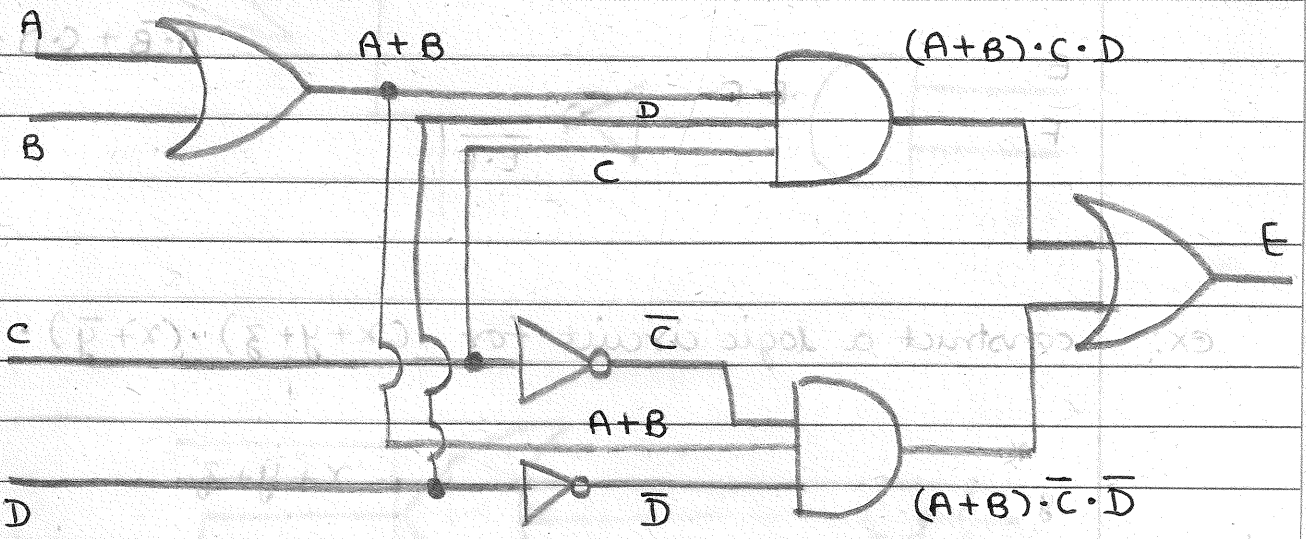


Page No.
Date

ex;



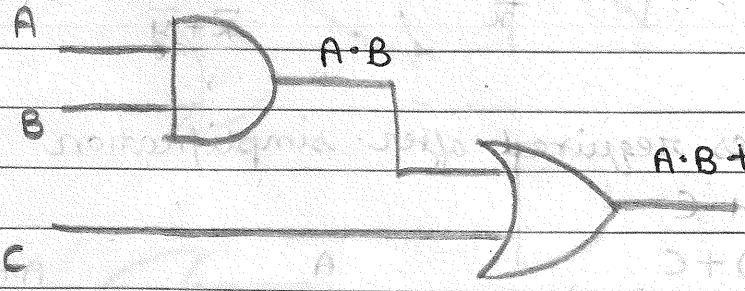
ex:



$$E = (A+B)\cdot C\cdot D + (A+B)\cdot \overline{C}\cdot \overline{D}$$

Converting expression to logic circuits

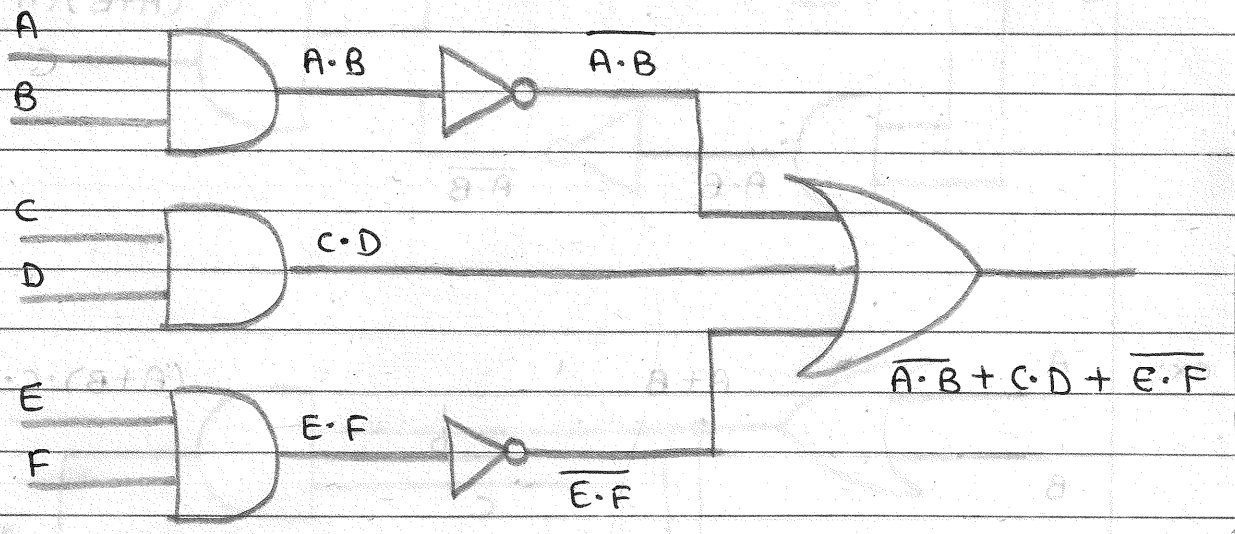
ex: construct logic circuit for Boolean expression $A\cdot B+C$



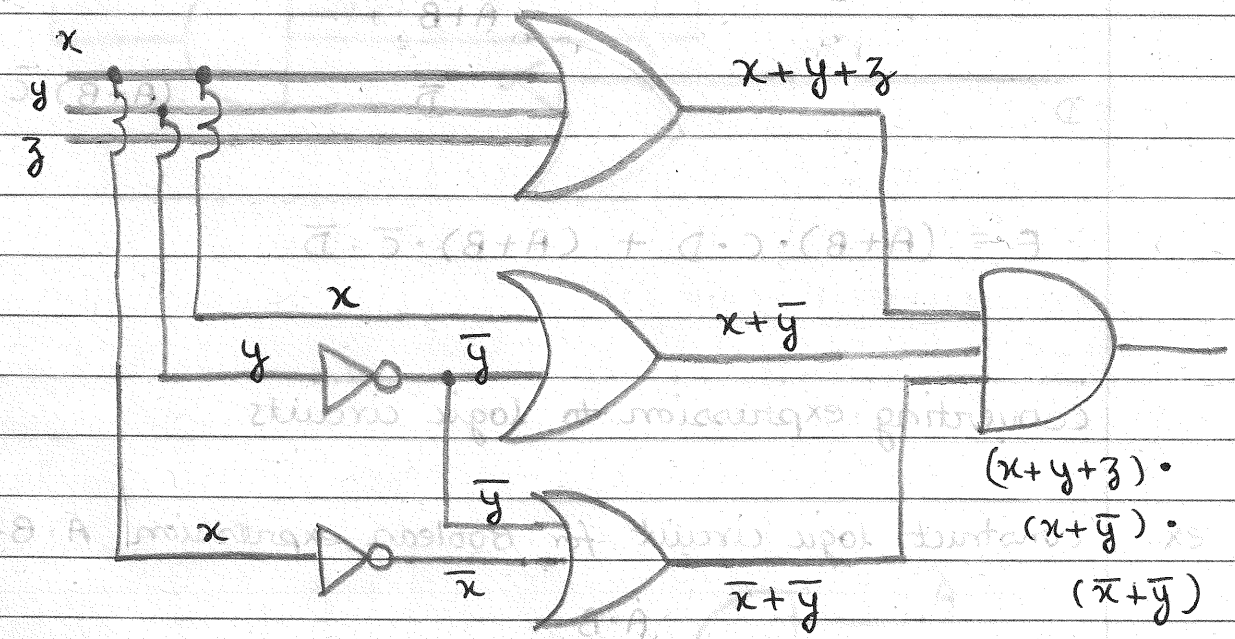
Page No.
Date

ex: construct a logic circuit for Boolean expression

$\overline{A \cdot B} + C \cdot D + \overline{E \cdot F}$



ex: construct a logic circuit for $(x+y+z) \cdot (x+\bar{y}) \cdot (\bar{x}+\bar{y})$

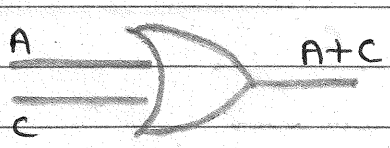


ex: no. of gates required after simplification

$A + A \cdot B + C$

$A(1+B) + C$

$A + C$

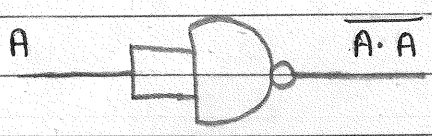


only one OR gate required after simplification.

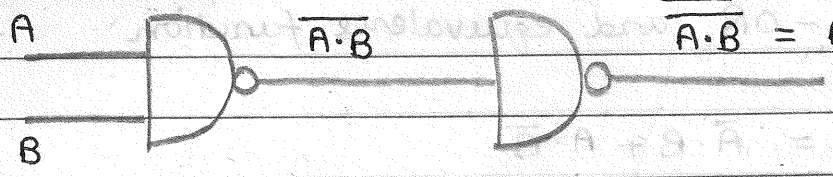
Page No.
Date

The universal nand gate.

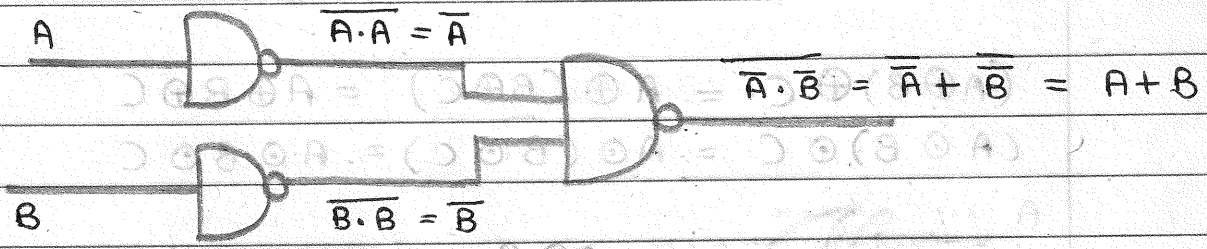
NAND is known as universal gate. because any gate can be implemented using NAND gate.



(a) NOT gate (requires one NAND gate)



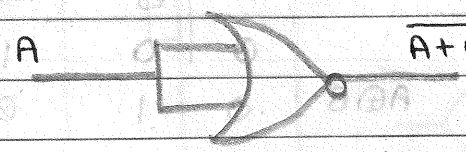
(b) AND gate. (require two NAND gate)



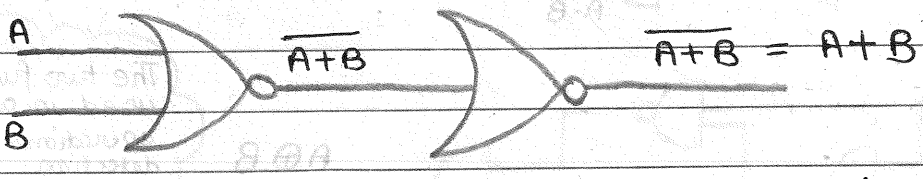
(c) OR gate. (require three NAND gate)

The universal NOR gate.

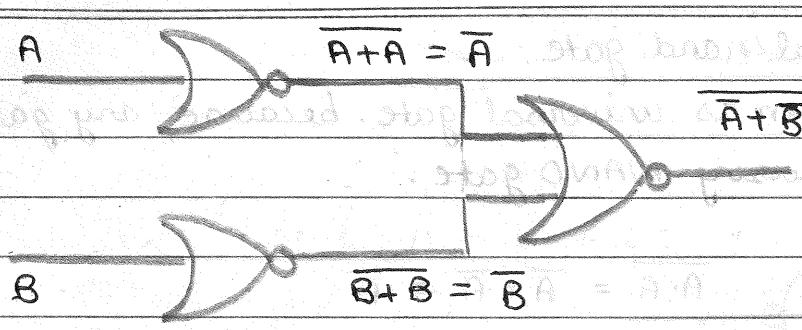
NOR function is dual of NAND function. NOR is also universal gate because it is sufficient to implement any Boolean expression.



(a) NOT gate (requires one NOR gate)



(b) OR gate (requires two NOR gate)



(c) AND gate (requires 3 NOR gate)

Exclusive - OR and equivalence function

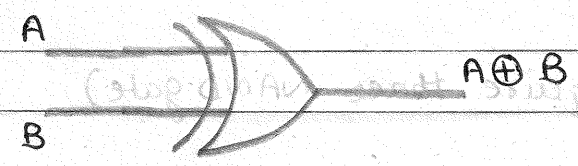
$A \oplus B = \overline{A} \cdot B + A \cdot \overline{B}$

$A \odot B = A \cdot B + \overline{A} \cdot \overline{B}$

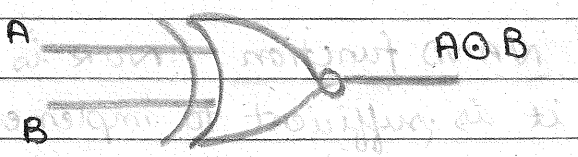
(two operations are complement of each other)

$(A \oplus B) \oplus C = A \oplus (B \oplus C) = A \oplus B \oplus C$

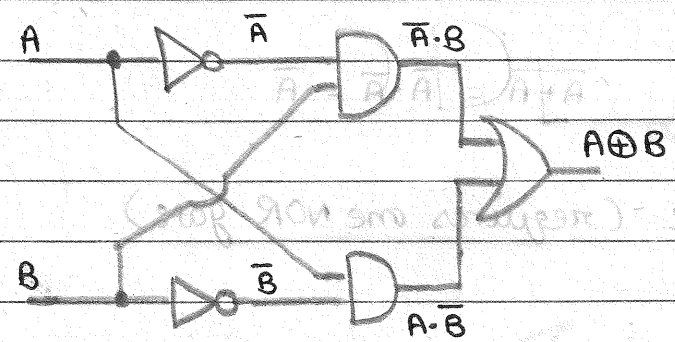
$(A \odot B) \odot C = A \odot (B \odot C) = A \odot B \odot C$



A	B	$A \oplus B$
0	0	0
0	1	1
1	0	1
1	1	0

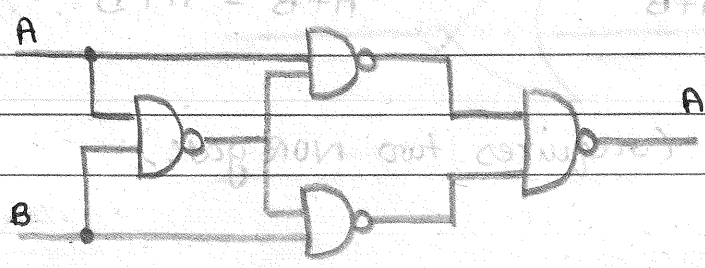


XOR → using NOT / AND / OR gate



A	B	$A \odot B$
0	0	1
0	1	0
1	0	0
1	1	1

XOR → using NAND gate.



The two func^n are used in arithmetic operations and error detection and correction.

Page No.
Date

Logic Family

A logic family of monolithic digital integrated circuit devices is a group of electronic logic gates constructed using one of several different designs, usually with compatible logic levels and power supply characteristics within a family

The main types of logic families are.

(i) TTL logic: (Transistor logic) → logic gates and other digital circuit are made using bipolar junction transistor & resistors.

(ii) ECL: (Emitter coupled logic), referred to as current mode logic families is a digital technology with extremely high speed.

(iii) CMOS (complementary metal oxide semiconductor) has complementary and symmetrical NMOS and PMOS transistor. Because of high noise immunity and low static power dissipation. CMOS is most preferred in large scale integrated circuits.

lowest propogation delay: ECL < TTL < CMOS
(ECL: min, CMOS: max)

minimum power dissipation: CMOS.
CMOS consumes less power than TTL.
ECL is fastest.

Page No.
Date

Logic Family

A logic family of monolithic digital integrated circuit devices is a group of electronic logic gates constructed using one of several different designs, usually with compatible logic levels and power supply characteristics within a family

The main types of logic families are.

(i) TTL logic: (Transistor logic) → logic gates and other digital circuit are made using bipolar junction transistor & resistors.

(ii) ECL (Emitter coupled logic), referred to as current mode logic families is a digital technology with extremely high speed.

(iii) CMOS (complementary metal oxide semiconductor) has complementary and symmetrical NMOS and PMOS transistor. Because of high noise immunity and low static power dissipation. CMOS is most preferred in large scale integrated circuits.

lowest propogation delay: ECL (min) < TTL < CMOS (max)

minimum power dissipation: CMOS.
CMOS consumes less power than TTL.
ECL is fastest.

Page No.
Date

Minimization of Boolean Expression → less number of boolean variables.

- Boolean Algebra Method
- Karnaugh Map (K-Map)

I) Karnaugh Map (K-map)

↳ composed of arrangement of cells each representing one particular combination on variables product form.
For total no. of n variables, k-map consist of 2^n cells.

(i) Four variable k-map. ($2^4 = 16$ cells) (A, B, C, D)

AB \ CD	00 $\bar{C}\bar{D}$	01 $\bar{C}D$	11 CD	10 $C\bar{D}$
00 $\bar{A}\bar{B}$	0	1	3	2
01 $\bar{A}B$	4	5	7	6
11 AB	12	13	15	14
10 $A\bar{B}$	8	9	11	10

(ii) Five variable k-map ($2^5 = 32$ cells)

↳ A, B, C, D, E

BC \ DE	$\bar{D}\bar{E}$	$\bar{D}E$	DE	$D\bar{E}$
00 $\bar{B}\bar{C}$				
01 $\bar{B}C$				
11 BC				
10 $B\bar{C}$				

A

BC \ DE	$\bar{D}\bar{E}$	$\bar{D}E$	DE	$D\bar{E}$
00 $\bar{B}\bar{C}$				
01 $\bar{B}C$				
11 BC				
10 $B\bar{C}$				

$\bar{A}$

Rules for grouping (group only 1), (not zero)

(i) group should be horizontal or vertical (not diagonally)
(ii) group should be 8, 4, 2, 1 (2^n cells) : group should be large as possible.
(iii) groups may overlap, may wrap around table, few groups only.

Page No.
Date

Q)

ab/cd	00	01	11	10
00	1	1		1
01				1
11				
10	1	1		1

(a) $C'D' + A'B' + BC$

(b) $AB + CD' + B'C$

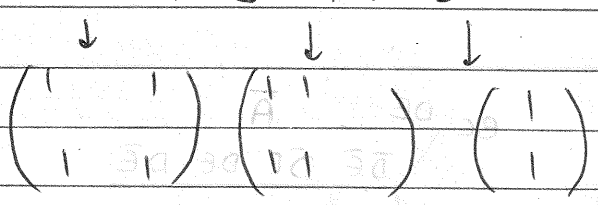
(c) $B + AC'$

(d) $B'D' + B'C' + A'CD'$

Soln.

	AB/CD	$\bar{C}\bar{D}$ 00	$\bar{C}D$ 01	CD 11	$C\bar{D}$ 10
$\bar{A}\bar{B}$	00	1	1		1
$\bar{A}B$	01				1
AB	11				
$A\bar{B}$	10	1	1		1

→ $B'D' + B'C' + A'CD'$



Boolean Algebra

Page No. | Date

Logic gates are building blocks of all logic circuits in a computer. Logic circuit are used in processor.

Boolean Algebra
George Boole developed algebra
→ provides an economical and straightforward approach to design of relay and other types of switching circuits.

Logical Addition → 'OR' operator (+)

Inputs		
A	B	A+B
0	0	0
0	1	1
1	0	1
1	1	1

Logical multiplication → 'AND' (·)

Inputs		
A	B	A·B
0	0	0
0	1	0
1	0	0
1	1	1

Complementation → 'NOT' (−)

Input	
A	$\bar{A}$
0	1
1	0

'OR' and 'AND' are binary operation because they operate on two variable

Complemation is unary operation defined on single variable.

Operator precedence.

(i) expression is scanned from left to right
(ii) bracket / parentheses are evaluated first
(iii) Complement (NOT) operation are performed next
(iv) AND operation after that (·)
(v) Finally, all OR in the end (+)

Page No.
Date

Postulates of Boolean Algebra.

(I)

		Involution law
$x \cdot \bar{x} = 0$	$x + \bar{x} = 1$	$\bar{\bar{x}} = x$
$x \cdot 1 = x$	$x + 1 = 1$	
$x \cdot 0 = 0$	$x + 0 = x$	
$x \cdot x = x$	$x + x = x$	

(II) commutative law

$x + y = y + x$

$x \cdot y = y \cdot x$

(III) associative law

$(x + y) + z = x + (y + z)$

$(x \cdot y) \cdot z = x \cdot (y \cdot z)$

(IV) Distributive law

$x \cdot (y + z) = x \cdot y + x \cdot z$

$x + (y \cdot z) = (x + y) \cdot (x + z)$

(V) Demorgan's law

$\overline{x + y} = \bar{x} \cdot \bar{y}$

$\overline{x \cdot y} = \bar{x} + \bar{y}$

↳ literal changes.

The Principle of Duality

interchanging '+' with '.' and '0' with '1' and vice versa. literal never changes itself.

(VI) Absorption law.

$x + x \cdot y = x$

$x \cdot (x + y) = x$ ← this holds by duality.

It is possible to enumerate all possible combinations of variable involved because Boolean Algebra deals with variables having only two values. This method of proving theorems is called exhaustive enumeration or perfect induction.

(VII)

$x + \bar{x} \cdot y = x + y$

$x \cdot (\bar{x} + y) = x \cdot y$

Page No.
Date

Boolean function.
↳ is formed with binary variables.

$$W = X + \bar{Y} \cdot Z$$

Here, W is a function of variables X, Y, Z. written as
$W = f(X, Y, Z)$
right hand side of equation is called an expression.
X, Y, Z are referred to as literals of this function.

no of rows in truth table will be equal to 2^n, where n is no. of literals in function.

Minimization of Boolean Functions.
Logic gates are used to implement Boolean function.
Each term of function is implemented with gate and each literal of term forms an input to gate.

ex: Simplify following Boolean functions to minimum no of literals.

(i) $F_a = x + \bar{x} \cdot y$
$= (x + \bar{x}) \cdot (x + y)$
$= 1 \cdot (x + y)$
$= x + y$

(ii) $F_b = x \cdot (\bar{x} + y)$
$= x \cdot \bar{x} + x \cdot y$
$= 0 + x \cdot y$
$= x \cdot y$

(iii) $F_c = (x + y) \cdot (\bar{x} + z) \cdot (y + z)$
$= (x + y) \cdot (\bar{x} + z) \cdot y$

(iv) $F_c = \bar{x} \cdot \bar{y} \cdot z + \bar{x} \cdot y \cdot z + x \cdot \bar{y}$

$= \bar{x} \cdot z \,(y + \bar{y}) + x \cdot \bar{y}$

$= \bar{x} \cdot z + x \cdot \bar{y}$

(v) $F_d = x \cdot y + \bar{x} \cdot z + y \cdot z$

$= x \cdot y + \bar{x} \cdot z + y \cdot z \cdot 1$

$= x \cdot y + \bar{x} \cdot z + y \cdot z \,(x + \bar{x})$

$= x \cdot y + \bar{x} \cdot z + x \cdot y \cdot z + \bar{x} \cdot y \cdot z$

$= x \cdot y + x \cdot y \cdot z + \bar{x} \cdot z + \bar{x} \cdot y \cdot z$

$= x \cdot y \,(1 + z) + \bar{x} \cdot z \,(1 + z)$

$= x \cdot y + \bar{x} \cdot z$

Complement of following function.

(1) $F_1 = \bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z$ Demorgan's law is used.

$\overline{F_1} = \overline{\bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z}$

$= \left(\overline{\bar{x} \cdot y \cdot \bar{z}}\right) \cdot \left(\overline{\bar{x} \cdot \bar{y} \cdot z}\right)$

$= \left(\bar{\bar{x}} + \bar{y} + \bar{\bar{z}}\right) \cdot \left(\bar{\bar{x}} + \bar{\bar{y}} + \bar{z}\right)$

$= (x + \bar{y} + z) \cdot (x + y + \bar{z})$

(ii) $F_2 = x \cdot (\bar{y} \cdot \bar{z} + y \cdot z)$

$\overline{F_2} = \overline{x \cdot (\bar{y} \cdot \bar{z} + y \cdot z)}$

$= \bar{x} + \overline{\bar{y} \cdot \bar{z} + y \cdot z}$

$= \bar{x} + (y + z) \cdot (\bar{y} + \bar{z})$

A simple procedure for deriving complement of function is (i) take dual of function
(ii) complement each literal.

ex: Find complement of function of F_1 and F_2 of previous example by their dual and complementing each literal.

(a) $F_1 = \bar{x} \cdot y \cdot \bar{z} + \bar{x} \cdot \bar{y} \cdot z$

dual $= (\bar{x} + y + \bar{z}) \cdot (\bar{x} + \bar{y} + z)$

complement $= (x + \bar{y} + z) \cdot (x + y + \bar{z})$

(b) $F_2 = x \cdot (\bar{y} \cdot \bar{z} + y \cdot z)$

dual $= x + (\bar{y} + \bar{z}) \cdot (y + z)$

complement $= \bar{x} + (y + z) \cdot (\bar{y} + \bar{z})$

Canonical form of Boolean Functions (Minterms & Maxterms)

Variable			minterm (m) → SOP (AND term of n variables)	Maxterm (M) → POS (OR term of n variable)
x	y	z		
0	0	0	$\bar{x} \cdot \bar{y} \cdot \bar{z}$	$x + y + z$
0	0	1	$\bar{x} \cdot \bar{y} \cdot z$	$x + y + \bar{z}$
0	1	0	$\bar{x} \cdot y \cdot \bar{z}$	$x + \bar{y} + z$
0	1	1	$\bar{x} \cdot y \cdot z$	$x + \bar{y} + \bar{z}$
1	0	0	$x \cdot \bar{y} \cdot \bar{z}$	$\bar{x} + y + z$
1	0	1	$x \cdot \bar{y} \cdot z$	$\bar{x} + y + \bar{z}$
1	1	0	$x \cdot y \cdot \bar{z}$	$\bar{x} + \bar{y} + z$
1	1	1	$x \cdot y \cdot z$	$\bar{x} + \bar{y} + \bar{z}$

How to remember: minterm gives priority to 1 and Σ
maxterm gives priority to 0 and π

Page No.
Date

Product of Sum : (POS) (π)
↳ several sum terms (ORs) (maxterms) logically multiplied (ANDed) together.

Sum of Product : (SOP) (Σ)
↳ several product terms (ANDs) (minters) logically added (ORed) together

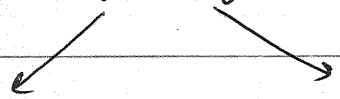
maxterm is complement of minterm
$\overline{m_j} = M_j$

Conversion b/w canonical form.
↳ to convert from one canonical form to another, interchange symbol (Σ with π or π with Σ) and list those numbers (from all possible numbers in range 0 to $2^n - 1$, where n is number of binary variable in func^n) missing from original form.

ex: $F(x, y, z) = \pi(0, 2, 4, 5)$ is in product of maxterm.
Its conversion to sum of minterm
$F(x, y, z) = \Sigma(1, 3, 6, 7)$

Page No.
Date

There are 2 types of circuit



sequential (clock) circuit	combinational circuit (no clock)
↳ (whose output depends not only on present state but also on what its input cond^n have been in past ie stored levels)	(circuit in which we combine different gates in circuit)
↓	eg: encoder, decoder, MUX, deMUX etc.
eg: Flipflops, counters etc.	(present input only).
(present + past) input	

Page No. | Date

Design of combinational circuits

Half adder: (addition of two bits)

↳ needs two binary input & two binary output.

Input		output	
A	B	C	S
0	0	0	0
0	1	0	1
1	0	0	1
1	1	1	0

$S = \bar{A} \cdot B + A \cdot \bar{B}$

$C = A \cdot B$

It cannot accept a carry generated from previous pair of lower significant bits. A full adder solves this problem.

Full adder: (addition of three bits)

↳ needs three binary input & two output output.

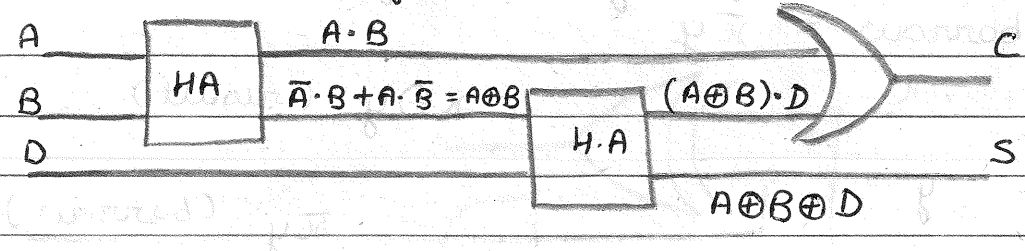
A	B	D	C	S
0	0	0	0	0
0	0	1	0	1
0	1	0	0	1
0	1	1	1	0
1	0	0	0	1
1	0	1	1	0
1	1	0	1	0
1	1	1	1	1

$S = \bar{A} \cdot \bar{B} \cdot D + \bar{A} \cdot B \cdot \bar{D} + A \cdot \bar{B} \cdot \bar{D} + A \cdot B \cdot D$

$C = \bar{A} \cdot B \cdot D + A \cdot \bar{B} \cdot D + A \cdot B \cdot \bar{D} + A \cdot B \cdot D$

or $= A \cdot B + A \cdot D + B \cdot D$

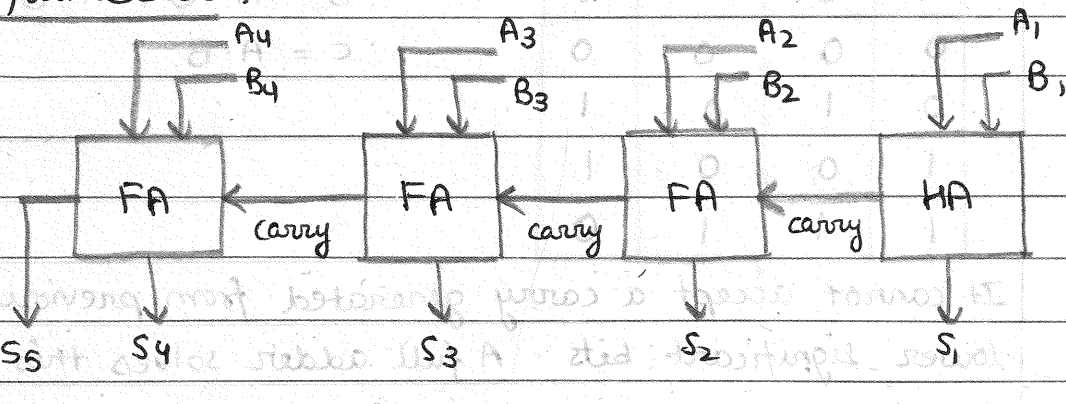
A full-adder can also be implemented with two half adder and one OR gate.



A parallel Binary Adder.

↳ add two binary numbers.

↳ if we want to add four-bit numbers, we need to construct a parallel four-bit binary adder. such an adder requires one-half adder and three full adder.



The capacity can be increased by connecting more full adders to left end of system.

ex: to add six-bit numbers, add 5 full adders and 1 half adder.

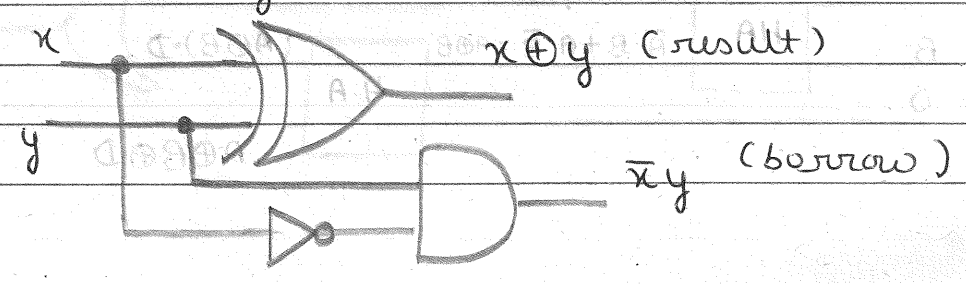
Subtractors

(1) Half subtractor: (two bit)

x	y	result	borrow
0	0	0	0
0	1	1	1
1	0	1	0
1	1	0	0

result $= \bar{x}y + x\bar{y} = x \oplus y$

borrow $= \bar{x}y$



Page No. | Date

(ii) Full Subtractor (three bits)

x	y	z	Difference	Borrow
0	0	0	0	0
0	0	1	1	1
0	1	0	1	1
0	1	1	0	1
1	0	0	1	0
1	0	1	0	0
1	1	0	0	0
1	1	1	1	1

$$\text{difference} = x \oplus y \oplus z$$

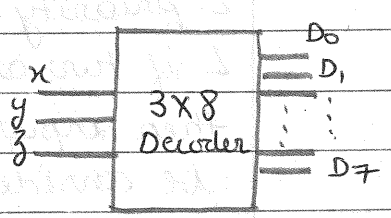
$$\text{borrow} = \bar{x}\bar{y}z + \bar{x}y\bar{z} + \bar{x}yz + xyz$$

(iii) Decoders.

Converts binary information from n-input lines of a maximum of unique 2^n output lines

A 3-bit binary to Decimal Decoder

Input x	y	z	output	minterm
0	0	0	D_0	$\bar{x}\bar{y}\bar{z}$
0	0	1	D_1	$\bar{x}\bar{y}z$
0	1	0	D_2	$\bar{x}y\bar{z}$
0	1	1	D_3	$\bar{x}yz$
1	0	0	D_4	$x\bar{y}\bar{z}$
1	0	1	D_5	$x\bar{y}z$
1	1	0	D_6	$xy\bar{z}$
1	1	1	D_7	xyz



If n-bit decoded information has unused or don't care combination, decoder will have less than 2^n outputs